

SQLite for Web Developers

Stack: Node.js, Express, EJS, better-sqlite3

Platform: Windows

Audience: Students who already have a working Express + EJS website and are ready to add persistent data storage

1. What is a database, and why do we need one?

Right now, when your Express server restarts, any data stored in a JavaScript array or object disappears. A **database** is a place to store data that survives restarts, and that can handle many users reading/writing at once.

Think of it like the difference between:

- A sticky note (JS array in memory) — gone when you close the laptop.
- A notebook (database file) — still there tomorrow.

Every real website you use — Instagram, a school portal, an online store — stores its data in a database, not in variables.

2. What is SQLite, and why we're using it

SQLite is a **relational database** that lives in a single file on disk (e.g. `database.db`). Unlike MySQL or PostgreSQL, it doesn't need a separate server process running in the background — no installing a "database service," no usernames/passwords/ports to configure. This matters a lot on Windows, where installing a full database server is a common source of setup headaches.

Key facts:

- Your entire database is one file. You can copy it, back it up, or delete it like any other file.
- It uses standard **SQL** (Structured Query Language), the same language used by MySQL, PostgreSQL, and virtually every relational database. What you learn here transfers directly.
- It's used in production by huge numbers of real apps (including as the embedded database inside browsers and phones) — it's not a "toy" database, just a simpler one to set up.

We'll use it through the npm package `better-sqlite3` , which is synchronous — meaning you don't need `.then()` , `await` , or callbacks to run a query. This makes the code much easier to read while you're learning.

3. Core SQL concepts

A SQLite database is made of **tables**. A table is like a spreadsheet: rows and columns.

Example `students` table:

id	name	grade
1	Sara	A
2	Ali	B

The four SQL commands you'll use constantly

Command	Purpose	Example
SELECT	Read data	<code>SELECT * FROM students</code>
INSERT	Add a row	<code>INSERT INTO students (name, grade) VALUES ('Sara', 'A')</code>
UPDATE	Change a row	<code>UPDATE students SET grade = 'A+' WHERE id = 1</code>
DELETE	Remove a row	<code>DELETE FROM students WHERE id = 1</code>

Creating a table

```
CREATE TABLE IF NOT EXISTS students (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  grade TEXT  
);
```

- `id INTEGER PRIMARY KEY AUTOINCREMENT` — every row needs a unique ID. SQLite generates this automatically.
- `TEXT NOT NULL` — a text column that can't be left empty.
- `IF NOT EXISTS` — safe to run every time the server starts; it won't wipe existing data.

Common column types

Type	Use for
INTEGER	whole numbers, IDs
TEXT	strings
REAL	decimal numbers
BLOB	binary data (images, files) — usually avoid; store a file path instead

4. Setting up better-sqlite3 in an Express project

Step 1 — Install

```
npm install better-sqlite3
```

Windows note: this package compiles a small piece of native code during install. If install fails, the fix is almost always to update to a recent Node.js LTS version (18+) — prebuilt binaries avoid needing Visual Studio Build Tools.

Step 2 — Create db.js

```
const Database = require('better-sqlite3');
const db = new Database('database.db');

db.exec(`
  CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    grade TEXT
  )
`);

module.exports = db;
```

This file does two jobs: it opens (or creates) `database.db`, and it makes sure the table exists. Every other file in the project will `require('./db')` to reuse this same connection.

Step 3 — Confirm it works

Run your server (`node app.js` or `nodemon app.js`). If `database.db` appears in your project folder, it worked.

5. CRUD: Create, Read, Update, Delete

`better-sqlite3` uses two main methods:

- `db.prepare(sql).run(...)` — for INSERT, UPDATE, DELETE (changes data, doesn't return rows)
- `db.prepare(sql).all()` — for SELECT, returns an array of rows
- `db.prepare(sql).get()` — for SELECT, returns just the first row (or `undefined`)

Create (INSERT)

```
app.post('/students', (req, res) => {  
  const { name, grade } = req.body;  
  db.prepare('INSERT INTO students (name, grade) VALUES (?, ?)').run(name, grade);  
  res.redirect('/students');  
});
```

Read (SELECT)

```
app.get('/students', (req, res) => {  
  const students = db.prepare('SELECT * FROM students').all();  
  res.render('students', { students });  
});
```

Get a single student by ID:

```
app.get('/students/:id', (req, res) => {  
  const student = db.prepare('SELECT * FROM students WHERE id = ?').get(req.params.id);  
  res.render('student-detail', { student });  
});
```

Update (UPDATE)

```
app.post('/students/:id/edit', (req, res) => {  
  const { name, grade } = req.body;  
  db.prepare('UPDATE students SET name = ?, grade = ? WHERE id = ?')  
    .run(name, grade, req.params.id);  
  res.redirect('/students');  
});
```

Delete (DELETE)

```
app.post('/students/:id/delete', (req, res) => {  
  db.prepare('DELETE FROM students WHERE id = ?').run(req.params.id);  
  res.redirect('/students');  
});
```

Teaching point: every one of these follows the same shape — `db.prepare(SQL).run/all/get(values)` . Once students recognize this pattern, adding new features to their own tables (posts, comments, products, etc.) becomes copy-paste-adapt rather than starting from scratch.

6. Connecting SQLite to EJS views

Data pulled from `.all()` is a plain JavaScript array of objects — EJS doesn't know or care that it came from a database.

`views/students.ejs :`

```

<!DOCTYPE html>
<html>
<head><title>Students</title></head>
<body>
  <h1>Student List</h1>

  <ul>
    <% students.forEach(student => { %>
      <li>
        <%= student.name %> – <%= student.grade %>
        <form action="/students/<%= student.id %>/delete" method="POST" style="display:inline">
          <button type="submit">Delete</button>
        </form>
      </li>
    <% }) %>
  </ul>

  <form action="/students" method="POST">
    <input name="name" placeholder="Name" required>
    <input name="grade" placeholder="Grade" required>
    <button type="submit">Add Student</button>
  </form>
</body>
</html>

```

Remember to enable form parsing in `app.js` :

```
app.use(express.urlencoded({ extended: true }));
```

Without this line, `req.body` will be `undefined` and every form submission will crash.

7. Data relationships (one-to-many, many-to-many)

Real apps almost always have more than one table, linked together. Example: students and courses.

One-to-many (a teacher has many students)

```
CREATE TABLE teachers (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL  
);  
  
CREATE TABLE students (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  teacher_id INTEGER,  
  FOREIGN KEY (teacher_id) REFERENCES teachers(id)  
);
```

`teacher_id` in the `students` table points back to a row in `teachers`. This is called a **foreign key**.

Querying with a join:

```
const students = db.prepare(`  
  SELECT students.name AS student_name, teachers.name AS teacher_name  
  FROM students  
  JOIN teachers ON students.teacher_id = teachers.id  
`).all();
```

Many-to-many (students enroll in many courses, courses have many students)

This needs a third "junction" table:

```
CREATE TABLE courses (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL  
);  
  
CREATE TABLE enrollments (  
  student_id INTEGER,  
  course_id INTEGER,  
  FOREIGN KEY (student_id) REFERENCES students(id),  
  FOREIGN KEY (course_id) REFERENCES courses(id)  
);
```

Teaching point: this is a good "advanced challenge" once students are comfortable with single-table CRUD — it's the same pattern behind features like "likes," "followers," or "tags."

8. Security: preventing SQL injection

Rule to teach on day one, not as an "advanced topic": never insert user input directly into a SQL string.

✗ **Never do this:**

```
db.exec(`SELECT * FROM students WHERE name = '${req.body.name}'`);
```

A user could type `' OR '1'='1` into a form and read the entire table, or worse.

✓ **Always do this instead — use `?` placeholders:**

```
db.prepare('SELECT * FROM students WHERE name = ?').get(req.body.name);
```

`better-sqlite3` automatically escapes the value safely when you use `?`. This single habit prevents the most common real-world web security vulnerability.

9. Debugging and inspecting your database

Install **DB Browser for SQLite** (free, Windows-friendly): <https://sqlitebrowser.org/>

- Open `database.db` directly to see tables and rows.
- Run raw SQL queries and see results instantly.
- Great for verifying "did my INSERT actually work?" without adding `console.logs` everywhere.

This tool is especially useful for demystifying what's happening "under the hood" of the `.run()` / `.all()` calls.

10. Common errors and fixes

Error	Cause	Fix
<code>SQLITE_ERROR: no such table</code>	Table wasn't created, or typo in table name	Check <code>CREATE TABLE IF NOT EXISTS</code> ran; check spelling
<code>req.body is undefined</code>	Missing form parsing middleware	Add <code>app.use(express.urlencoded({ extended: true }))</code>
App crashes on <code>npm install</code>	Old Node.js version, missing build tools	Update Node.js to latest LTS
Data disappears after restart	Table recreated with <code>DROP TABLE</code> accidentally, or wrong file path	Check <code>CREATE TABLE IF NOT EXISTS</code> (not <code>CREATE TABLE</code> alone); confirm same <code>.db</code> filename used everywhere
<code>UNIQUE constraint failed</code>	Trying to insert a duplicate value in a column marked <code>UNIQUE</code>	Check for existing row before inserting, or handle the error
Changes don't show on page	Forgot to <code>res.redirect()</code> after a POST, or browser cache	Add redirect after insert/update/delete

11. Practice exercises

Give these in order — each builds on the last.

1. **Setup:** Create a `books` table (`id` , `title` , `author` , `year`). Seed it with 3 books using `db.exec()` .

2. **Read:** Build a `/books` route that lists all books in an EJS page.
3. **Create:** Add a form to submit a new book. Make sure the list updates after submitting.
4. **Delete:** Add a delete button next to each book.
5. **Update:** Add an edit form (pre-filled with the current values) for each book.
6. **Validation:** Prevent submitting a book with an empty title (check both in the HTML form with `required` and in the Express route).
7. **Search:** Add a search box that filters books by title using `WHERE title LIKE ?`.
8. **Relationships:** Add an `authors` table and link `books.author_id` to it instead of storing the author name as text.
9. **Security check:** Try to break your own app with a SQL injection attempt (e.g., typing `'` or `'1'='1` into a form) and confirm it doesn't work.

12. Mini project: build a full CRUD app

Assign this as a capstone once the exercises are done: **a task manager** with the following table:

```
CREATE TABLE tasks (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL,  
  done INTEGER DEFAULT 0  
);
```

Requirements:

- List all tasks, with completed ones visually distinct (e.g., strikethrough).
- Add a new task via a form.
- Toggle a task's `done` status with a button.
- Delete a task.
- Bonus: add a "clear completed" button that deletes all tasks where `done = 1` in one query.

This single project touches every CRUD operation and reinforces the request → route → SQL → response → EJS render cycle end to end.

13. Reference cheat sheet

```
// Setup
const Database = require('better-sqlite3');
const db = new Database('database.db');

// Create table
db.exec(`CREATE TABLE IF NOT EXISTS items (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL
)`);

// INSERT
db.prepare('INSERT INTO items (name) VALUES (?)').run(name);

// SELECT all
db.prepare('SELECT * FROM items').all();

// SELECT one
db.prepare('SELECT * FROM items WHERE id = ?').get(id);

// UPDATE
db.prepare('UPDATE items SET name = ? WHERE id = ?').run(name, id);

// DELETE
db.prepare('DELETE FROM items WHERE id = ?').run(id);

// Search (partial match)
db.prepare('SELECT * FROM items WHERE name LIKE ?').all(`%${searchTerm}%`);

// Join
db.prepare(`
  SELECT a.name AS a_name, b.name AS b_name
  FROM a JOIN b ON a.b_id = b.id
`).all();
```

Golden rules to repeat often:

1. Always use `?` placeholders — never build SQL with template literals from user input.
2. `.run()` for changes, `.all()` for many rows, `.get()` for one row.
3. `CREATE TABLE IF NOT EXISTS` so restarting the server never wipes data.
4. One `db.js` file, `require` d everywhere you need it — don't open multiple connections.